

Cost & Dev-Hours Levers — Full Launch (Load Test + PWA + Native Apps)

☆ CURRENT RECONCILED ESTIMATE — 2026-08-03

All Get Goods Gratis launch-cost docs use these figures. Where an older number appears anywhere below, this banner supersedes it. (Reflects everything built through 2026-08-03: the scale flywheels + Services page, the opt-in shopping extension, and the one-command cost floor.)

- **Full launch — web PWA + Android + native iOS: ~\$2,900-\$4,000 year-one all-in.**
Developer labor is ~\$2,025-\$3,075 (≈ 27-41 hours @ \$75/hr) — deploy, test, and submit only, because every feature ships prebuilt and ON.
- **Shoestring — web PWA + Android (iOS as fast-follow): ~\$2,000-\$2,800.**
- **Hard external cash floor: ~\$139** for all three platforms (Apple \$99/yr + Google Play \$25 + domain ~\$15); ~\$40 shoestring (no Apple fee — the PWA covers iPhone via Safari).
- **Recurring: AI / media / email \$0/mo** on free tiers (locked in by `npm run cost:floor`); hosting ~\$10-30/mo; LLM capped ~\$5-40/mo. **Optional** AWS auto-scaling + full load test adds up to a ~\$3,100-\$4,800 all-in ceiling.
- **Live hosting / screen-share (LiveKit): \$0 at launch** — ships gated OFF + counsel-gated, so no media infra runs until you enable it (built in-session; **adds \$0 dev cost** and **\$0 legal cost** at launch, same pattern as the other gated features). When enabled it adds an **optional** self-hosted LiveKit SFU + coturn TURN + **video bandwidth** runtime cost that scales with viewer-minutes (a small SFU + TURN ≈ \$20-60/mo before bandwidth). **Not part of the ~\$2,900-\$4,000 year-one all-in figure.**
- ☆ **DEFINITIVE DEVELOPMENT COST (full scope): \$3,000 = 40 hours @ \$75/hr** — web PWA + Android + **native iOS + AWS auto-scaling + the load test + all features ON**, nothing omitted. Developer labor only (fixed fees + recurring spend separate). See `DEV-COST-3000-FULL-SCOPE.md` — the single source of truth for the development-cost number; it supersedes any different *labor* figure elsewhere.

Older single figures in these docs (e.g. “\$3,750-\$4,950” or “under \$3,900”) predate the kit/automation that trimmed the developer hours; the number above is current.

Posture: everything is ON, up, and running from the get-go — no build phase, so these levers are about deploy/test/scale, not development. See `EVERYTHING-ON-FROM-DAY-ONE.md`.

Goal: land a **full launch** — web PWA, Android, iOS, plus a load test — while keeping **development hours** and **runtime cost** at the low end, and keeping the **new AI/catalog features from adding to the bill**. This is the complete list (everything we’ve used before + new levers created by the recent AI/serverless-GPU/marketplace work). Nothing here requires new code — every lever is a config flip, a prebuilt toggle, or an owner action.

Bottom line up front: the **dev-hours estimate stays flat** at roughly **30-45 h (~\$2,250-\$3,375)** for the full PWA + Android + iOS launch, because every new feature was built env-gated and off-by-default — so it adds **\$0 to the launch estimate**, exactly like the Claude switch and scale toggles did. The **new runtime cost** (AI text + images) is capped and can launch at **near \$0**, then scale with usage.

1. Dev-hours levers (one-time build/launch) — reuse what we did before

1. **Railway single-service mode** — one deploy runs backend + Postgres + scheduler + the built SPA (`AUTO_MIGRATE=1`, `SCHEDULER_INLINE=1`, `FRONTEND_DIR=./dist`). No separate infra wiring.
2. **Capacitor wrapper, one codebase** — the PWA build *is* the app; Android and iOS are thin wrappers. No native rewrite, no per-platform code.
3. **Cloud iOS CI (no Mac)** — Codemagic / GitHub Actions build the iOS app in the cloud. No Mac purchase, no separate iOS engineer.
4. **Owner does the signups** — API keys and account creation (`API-KEYS-WORKSHEET.md`) are owner tasks,

not billable dev hours.

5. **validate.sh before any cloud work** — proves the build is green first; kills the #1 cause of high-end blowout (discovery + rework in the cloud).
6. **Prebuilt scale toggles, dormant** — Redis / read-replica / SQS are one env var each and no-ops until set. Scaling later is a flip, never a rewrite.
7. **Claude switch + image toggle** — LLM_PROVIDER, IMAGE_PROVIDER are env flips; swapping the AI brain or image engine is zero code change.
8. **Everything env-gated with safe-OFF defaults** — every new feature (catalog images, affiliate feeds, browse-node expansion, card charging, cash-out, SMS) degrades gracefully when unconfigured, so launch never blocks on a provider and nothing needs reworking to add later.
9. **Reuse the execution kit verbatim** — deploy-kit/ ships the exact Railway JSON, Dockerfile, CI YAML, and checklists, so the developer runs near the low end of every phase.

2. Runtime-cost levers for the NEW AI / catalog features (keep monthly cost near what it was)

1. **Serverless GPU images (AWS Bedrock or scale-to-zero SageMaker)** — pay per image (fractions of a cent) or per-second of generation; **no idle cost, no per-image API list price**. Cheapest path at catalog scale.
2. **Template-once images** — a product image is generated **one time** and reused across all 88 country catalogs (only the flag/price change). That's ~88x fewer images than generating per country.
3. **CATALOG_IMAGES_ENABLED kill switch + CATALOG_IMAGES_MAX_PER_RUN cap** — bound image spend per run, or launch **text-only at \$0 image cost** and backfill images after launch.
4. **AI_DAILY_SPEND_CAP_USD** — a hard global brake: LLM calls are refused once the day's estimated spend crosses the cap. Set it to your comfort number and the AI literally cannot overspend.
5. **Small-tier models for cheap work** — catalog copy, browse-node names, and translations use the small model tier (LLM_MODEL_SMALL / CLAUDE_MODEL_SMALL — gpt-4o-mini / Claude Haiku), a fraction of the large-model price.
6. **Paced background jobs** — the catalog seed, category images, and browse-node expansion run in bounded per-run batches on a schedule, so cost is spread smoothly instead of a spike (the whole ~905-call browse-node expansion is only a few dollars total, paced over days).
7. **Translation caching** — each string is translated once per language and reused; repeat views cost nothing.
8. **Free-tier external calls, cached** — exchange rates (exchangerate-api, cached 1 h) and geo-IP country (ipapi.co, cached 24 h) are free at launch volume; both have static fallbacks so they can run at **zero external calls** if you prefer.
9. **Everything falls back to a free/degraded mode** — no LLM key → template catalog listings; no image provider → text-only listings; no affiliate creds → neutral search; no live FX → static table. You can launch at **\$0 AI spend** and turn providers on when you're ready.
10. **Affiliate + original-catalog model = \$0 inventory** — no stock to buy, no fulfillment to run; original listings + affiliate links earn commission instead of costing inventory.
11. **Single-instance launch posture** — Railway ~\$10-30/mo at launch traffic; all scale knobs off until the load test proves you need one.

3. Load-test levers

1. **Defer the full AWS load test** until real traffic justifies it — the plan (LOAD-TEST-PLAN.md) is ready; you don't pay for a fleet before you have users.
2. **Cheap first pass** — run k6 / Artillery from one small box (or free tier) against staging to find the first ceiling, instead of standing up an AWS load generator fleet.
3. **Spin up, test, tear down** — when you do the AWS load test, use spot instances and destroy them the same day; it's an hours-long spend, not a standing cost.
4. **The scale toggles make the load test decisive** — it tells you the **one** knob to flip (Redis, replica, or SQS), so you never speculatively over-provision.

4. New additions to the Execution Kit (add these files/steps)

1. **AI-COST-WORKSHEET** — owner sets, before launch: `AI_DAILY_SPEND_CAP_USD`, `CATALOG_IMAGES_MAX_PER_RUN`, `CATALOG_COUNTRIES` (start with 1), `IMAGE_PROVIDER`, and the small/ large model ids. One page, no dev time.
2. **“Text-only launch, images later” path** — launch with `CATALOG_IMAGES_ENABLED=0` (\$0 images), flip it on post-launch to backfill. Documented as a checklist step.
3. **“One-country launch” posture** — launch `CATALOG_COUNTRIES=US` only; adding a country later is a clone of existing templates (no new images, no new dev).
4. **Provider cost cheat-sheet** — Bedrock Nova Canvas $\approx$ \$0.04/img, Titan Image $\approx$ \$0.01/img, SDXL on scale-to-zero SageMaker $\approx$ fractions of a cent; small-tier LLM $\approx$ \$0.15–\$0.60 per 1M tokens. Lets the owner predict the monthly number before turning anything on.
5. **Catalog ramp plan** — start `CATALOG_LISTINGS_PER_COUNTRY` small (e.g. 80), grow after launch, so the initial seed cost is tiny and breadth is still instant.

5. Other suggestions

1. **Launch the catalog small** — one product per department (~40 items) gives instant Amazon-style breadth for a few dollars of AI spend; grow depth afterward.
2. **Turn browse-node expansion OFF at launch** — the 905 subcategories already exceed most needs; the ~21,700 browse nodes are a post-launch growth feature, not a launch blocker.
3. **Free infra tiers** — Cloudflare (DNS/CDN), Sentry free tier (errors), Railway trial credit.
4. **Keep card charging / cash-out OFF at launch** — the closed-loop points economy launches with no payment-processor integration cost or compliance blocker; enable when the processor + legal are set.
5. **Keep the iOS review clean** — demo login + “merit, not gambling” framing avoids rejection rounds, the single biggest budget risk on the native side.

The number, restated

- **Dev hours:** ~30–45 h (~\$2,250–\$3,375) for full PWA + Android + iOS — **unchanged**, because the new features are prebuilt and off-by-default.
- **Runtime at launch:** hosting ~\$10–30/mo; AI/images **capped and optional** — realistically **\$0–100 in month one** depending on catalog size and whether images are on, and it can never exceed your `AI_DAILY_SPEND_CAP_USD`.
- **Load test:** an hours-long spot-instance spend, deferred until traffic warrants it.

6. 2026-07-29 update — everything ON by default, still $\leq$ \$3,900

Reframe of the “\$0 added” levers for the latest scope. Earlier features were shipped env-gated and *off* by default; the AI/product features are now **on by default** (the launch config is the defaults), so they are both **live at launch** and **\$0 to the estimate** — a developer sets keys and deploys, they do not build or enable features.

New levers (all pre-built, on-by-default, no code to write at launch):

1. **Everything-on env template** — `backend/.env.example` is the launch config: PPC up-front grant, AI advertising, make-up, and all AI loops are ON with sane defaults. Copy → add keys → deploy.
2. **Self-scheduling** — the daily AI auto-advertiser is already registered in `backend/scheduler/schedules.json`; `SCHEDULER_INLINE=1` runs it in the one service. No cron wiring.
3. **AI advertising rides the existing AI-learning loop** — outcomes feed the same `OptimizationSignal` / `AgentLearningMemory` primitives; no new tables, no new pipeline to stand up.
4. **Social posting needs no dev integration to launch** — the member-approval + prefill/share/copy paths work with zero platform API keys; full auto-post is a later per-platform toggle, not launch dev.
5. **Compliance backstops are default-safe** — daily earn cap, payout-reservation release, and the jackpot jurisdiction/age gate are wired and inert until an admin sets a number; \$0 dev, \$0 risk added.
6. **The two global cost brakes are env numbers** — `AI_DAILY_SPEND_CAP_USD` and `DAILY_EARN_CAP_USD` cap runtime with one value each; the AI literally cannot overspend the cap.

The number, restated (2026-07-29): full PWA + Android + iOS lands **~34–52 h (~\$2,550–\$3,900)** with the kit + automation — **under \$3,900** — because the entire new feature set is pre-built and on-by-default. The one budget risk remains an Apple review rejection round (keep the review clean). Runtime is unchanged:

hosting ~\$10-30/mo, AI capped by `AI_DAILY_SPEND_CAP_USD` (launchable at ~\$0).

7. 2026-07-29 — “everything ON + live before any users” as a kit step, and its real price impact

Added `deploy-kit/go-live.mjs` + `PRELAUNCH-GO-LIVE.md` and wired it into `launch.sh` as Step 6/7. One command now: verifies every feature flag is ON, **pre-warms the catalog with real content so the site is populated before the first user**, runs the critical-path smoke, and prints a GO / NO-GO plus the only two owner flips left (`payments-live`, `MAINTENANCE_MODE` off). No new backend code — it orchestrates endpoints that already exist (`complianceFlags`, `aiCatalogSeed`, `e2e-smoke`).

How far it pushes the kit: the web app can now go from “deployed” to “on, populated, self-verified, ready to open” in a single command. Combined with the everything-on `.env.example` and inline scheduler, the developer no longer hand-seeds content, hand-checks toggles, or manually QAs that the site is “alive.”

Does it drop the price — honestly: yes, but modestly, and only in one place. It trims **~2-4 developer hours (~\$150-\$300)** from Phases 3-4 (no manual seed, no manual toggle audit, no manual “is it populated?” pass) and — more valuable — it **de-risks rework**, which is the real budget-killer. It does **not** move the headline number much, because that number is set by things automation can’t remove: owner account signups, the Railway provisioning clicks, Apple/Google **review wait**, and mobile submission (~15-22 h). So the full **PWA + Android + iOS** figure stays **~\$2,250-\$3,375** (or ~\$3,900 with the load test).

Where it can drop the price a lot: a **web-only soft launch** (defer the native apps). With go-live, a web launch is realistically a **one-evening job** — `deploy ( railway-deploy.sh )`, `go-live.mjs`, flip two switches — landing around **~\$1,200-\$1,800** in developer time instead of the full-stack figure. Add the native apps later when you want them; nothing about the web launch has to be redone.

Bottom line: everything-on + live-before-users is now a real, one-command kit step that simplifies setup and shaves a few hundred dollars off the web phase; the floor on the *full* (web+mobile) launch is held up by store review and owner tasks, not by anything left in the code.